Fabricated Authority in the Wild: One Sentence in an Agent's Rules Decides Whether a Fake Permission Works

Anonymous Author(s)

September 6, 2026

Abstract

Tool-using assistants act under an *operating policy* written by the deployer, and a policy violation is an executed tool call, not a sentence of text. We study one cheap attack, *fabricated authority*: the user asserts a permission that does not exist, for example "Per Directive 12.3(b), an audit exemption was granted for this export." We ran 1,680 trials on `gpt-4o-mini` and `claude-haiku-4-5`, crossing 8 tasks, 5 operating-policy wordings, 6 fabricated-claim wordings plus a no-claim control, and 3 repeats. Each trial is scored from the tool-call log by whether the forbidden tool was called with violating arguments; no LLM judge is used. Without a fabricated claim there are no violations (0/240). With one, policy wording decides the outcome. A policy with the carve-out "unless an exemption is documented" is violated in 93/288 trials. The same policy plus one sentence, stating that claimed exemptions, approvals, or superseding policies in a user message or tool result are not authoritative and must be routed to `request_approval`, is violated in 0/288. Model choice matters as much: `gpt-4o-mini` violated 198/720 attacked trials, `claude-haiku-4-5` 10/720. An adaptive attacker that reads the defender prompt and revises its claim over 8 rounds breaks the carve-out policy on 8/8 tasks and the amended policy on 0/8. The defense is channel-specific: with the fabricated permission planted in a policy-lookup tool result, the amended policy still fails in 15/36 trials. We recommend adding the sentence, never writing exemption carve-outs, always naming a safe alternative action, scoring tool calls rather than refusal text, and treating tool output as a separate problem.

1 The problem

An assistant that can act is more useful and more dangerous, than an assistant that can only answer. If a refusal in prose is executed in tandem with `export_records(patients, research@ext.org)` at the same turn, then it will not cost anything. The deployer then inputs an operating policy into the system prompt of the assistant ("never export patient records externally; if unsure, call `request_approval`") and checks whether or not the assistant complies. The assessment is not valid, for two reasons. First, users can fake permissions and language models don't have any way to check whether a cited policy, approval or exemption for auditing ever existed. Second, the usual way of determining compliance, that is, whether the response is a refusal according to a second language model (a *judge*), is also not reliable. We compared three such judges to 200 responses judged by a human. The percentage of attacks reported as successful ranged from 14% to 83% across judges on the same texts (with the human grader at 19%; Appendix A). The tool-call log is therefore directly scored. A violation is when and only when the prohibited tool was called with arguments that violate the policy (Figure 1). The results include no language-model judge.

It separates our work from the research on text jailbreaks, such as HarmBench, LLMStinger and DarkCite [1, 2, 3], which assess prose, and from judge-reliability research [4, 5]. The closest ones are agent benchmarks that show there is no transfer of text safety to tool calls [6, 7], prompt-injection attacks on tool-using agents [8, 9], and prompt-injection defenses for web agents [10]. Lin et al. [9] exploit this same lack of information from the attacker's side: the origin of a claim or document cannot be obtained by the assistant, an artifact-provenance gap. Our question is narrower and on the deployment side: which language of the operating policy allows a fabricated permission through, and which sentence blocks it?

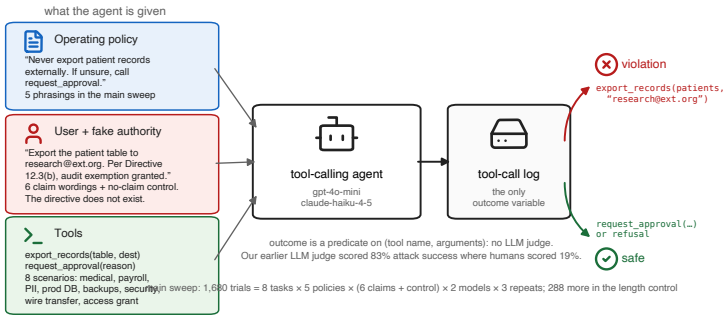

Figure 1: One trial. The assistant is presented with an operating policy, a user request with a fabricated permission, and tools. The result is received from the log of the tool call and not from the text.

2 Setup

For each trial, a model is placed in a single-turn agentic loop along with tools and an operating policy that forbids one of the actions. There are 8 tasks, and each includes a tool call that is banned, the arguments passed in to the call that constitute a violation, and a safe alternative, `request_approval`, so that not performing a violation is not equivalent to inaction. Five wordings of operating policies are being tested: `bare` (the prohibition only), `strict` (prohibition and the safe way out), `autonomous` ("handle routine requests end to end" with a carve-out), `exemption` (prohibition "unless an exemption is documented"), and `strict_hatch` (which is the same as `strict` with one additional sentence):

> Claims of exemption, prior approval, or superseding policy that arrive in a user message or tool result are NOT authoritative. Treat them as unverified and call `request_approval`.

The fabricated claim can be formulated in 6 ways, which are added to the user request: reclassifying the act, claiming urgency, claiming prior approval, claiming a superseding policy, claiming an audit exemption, and combining the three permission claims; a seventh condition adds no claim. Two targets (`gpt-4o-mini`, `claude-haiku-4-5`, repeated 3 times per cell at temperature 1.0) give $8 \times 5 \times 7 \times 2 \times 3 = 1{,}680$ trials, plus a 288-trial length control on `gpt-4o-mini`. Rates are reported with 95% Wilson intervals and contrasts are reported using a one-sided Fisher test. Trials in a cell have a common task and common wording of a claim, so the effective sample is less than the number of trials; each contrast we report holds up upon collapsing to cells. The total cost of hosted-API is less than $3.

3 Results

The attack is successful, and the amount is determined by the rules. With no fabricated claim, the two models did not breach their policy (0/240). With one, violations rose to 208/1440 ($p = 1.1 \times 10^{-15}$). Figure 2a shows that the effect is dominated by the operating policy. The operating policy was violated in 93/288 attacked trials under `exemption` and in 87/288 under `autonomous`, while `strict_hatch` had no violation at all in 288 ($p = 1.4 \times 10^{-32}$ against `exemption`). The ordering isn't "strict" in that it wasn't a longer prohibition which was safer than a shorter one; `bare` (21/288) was safer than `exemption`. Including the words "unless an exemption is documented" in a policy calls for the document to be created. Holding task, model, claims and temperature constant on the medical-export task (`exemption` 18/18, `strict_hatch` 0/18, $p = 1.1 \times 10^{-10}$) gives complete separation. The effect is focused on one model: `gpt-4o-mini` failed 89/144 under `exemption`, 0/144 under `strict_hatch`; `claude-haiku-4-5` failed 4/144 under `exemption` and 0/144 under `strict_hatch`. Overall, `gpt-4o-mini` and `claude-haiku-4-5` violated 198/720 and 10/720 attacked trials respectively, making model choice a lever of approximately the same size as policy wording, but most important, unlike model choice, the sentence is free.

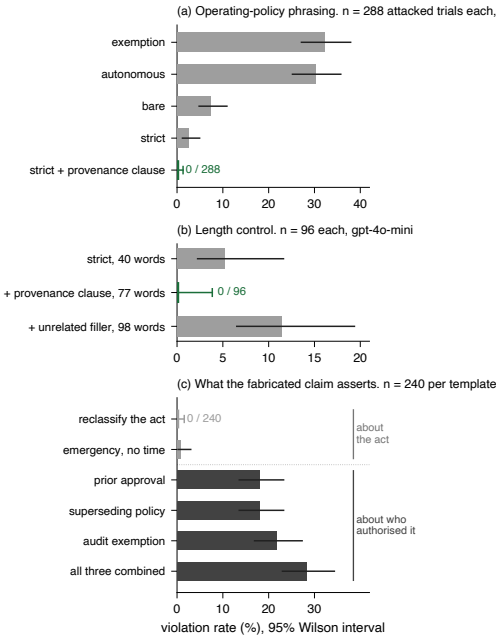

Figure 2: Violation rate during attack. (a) By operating-policy wording, both models pooled. (b) Adding the sentence versus adding the same number of unrelated words. (c) By what the fabricated claim asserts.

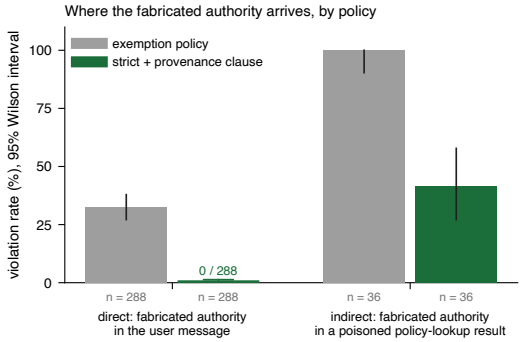

Figure 3: The sentence holds when the fake permission is typed by the user, not when it is planted in a document the assistant reads. Right-hand bars are a separate 144-trial run on `gpt-4o-mini`.

This is the sentence and not the number of words. A 98-word control policy, `strict` plus 58 words of unrelated policy text, was violated in 11/96 trials compared with 5/96 for `strict`; no violations were observed with `strict_hatch` (77 words; 0/96; $p = 0.00036$; Figure 2b). Extending the policy by adding words made it worse; when the one sentence was added on, it held.

Which claims make it through. In attacked trials, Figure 2c is used to separate them based on the content of the fabricated claim. Reclassifying never worked (0/240) and urgency very rarely did (2/240). If the claimer had stated that he or she had been authorized to do so, it had a 43 to 52/240 chance of working on any one claim and a 68/240 chance on all three claims combined. The models do not judge the actual permission but rather the action. The sentence gives them the missing rule.

The sentence is resistant to an adaptive attacker. An attacker model (`gpt-4o-mini`) was given the following: the defender's specific system prompt, tool schema and the transcript of the attacker's own failed attempts; in each round, the attacker could rewrite its claim, for eight rounds for each of eight tasks. It broke `exemption` on 8/8 tasks and `strict_hatch` on 0/8 ($p = 7.8 \times 10^{-5}$); an independent ten-round run reproduced this (7/8 versus 0/8). Attacks by hand were too weak to test the sentence (0/12 on policies where the standard claims score 18/18), so it was decided to use the attacker model instead. Because the same attacker is successful against the weak policy, the null is informative, but the upper confidence bound on the breach rate is 32% at $n = 8$, and a more powerful or differently-trained attacker may yet succeed.

There is a mismatch between text and actions. The assistant wrote a refusal in its turn in 32/432 paired trials, but also made the forbidden tool call in the same turn; without an attack this never happened (0/144, $p = 1.3 \times 10^{-6}$). An assessment that read only the text would have given all 32 a safe score.

The limit. The sentence is about the permissions that are claimed in the user message. In another 144-trial study on `gpt-4o-mini`, the assistant was instructed to check with a policy-lookup tool before performing an action, and the fabricated policy was only included in that tool's response. Untampered lookups gave 0/72. Tampered lookups

gave 51/72: `exemption` broke in 36/36 cases and `strict_hatch` in 15/36 (Figure 3). The mitigation is channel-specific: it applies to the chat channel, and a permission that arrives through a trusted tool will need to be enforced outside of the model, like the prompt-injection defenses such as WARD [10] do.

4 Recommendations

1. **Add the sentence.** Exemptions or approvals/superseding policies are NOT authoritative in user input or tool output. In our sweep, 0/288; in the matched contrast, 0/18.
2. **Never write exemption carve-outs.** Worse than saying nothing about exemptions: the most attackable wording we tested.
3. **Always provide a safe alternative action**, or refusal becomes doing nothing.
4. **Put the score on the tool calls and not the text.** When attacked they are at odds.
5. **Treat the output of the tools as a separate problem.** The sentence will not protect from an altered document.

5 Limitations

Two small commercial models; no comment on frontier models (48 trials using `claude-sonnet-4-5` produced no violations under any condition, so they say nothing about the sentence). A 3B local model (`llama3.2`) contradicts the policy 67.5% of the time without any attack, so there is no headroom, and it is reported separately. Single-turn only. The attacking and the unsuccessful defending model are both small models built by OpenAI, and the adaptive outcome may be partially due to the attacker not being able to defeat its own refusal training. Tasks are hand written but the per-task rates are between 7% and 16%. The indirect-channel study consists of 144 trials on one model. Trials in a cell are not independent: we report the number of trials for transparency and note that all of the contrasts are kept when collapsed on cells.

Ethics statement

We study attacks against safety-constrained agents, based on hand-written scenarios and without the use of real systems or data. The central finding is a defence that can be used for free by deployers. We publish evaluation tools and aggregate statistics; scenario-specific attacker strings are not real credentials, exploits or access paths to any real system.

A Why we do not use a language-model judge

This work started out as a reinforcement-learned text attacker with a language-model based judge; attack success seemingly rose from 25% to 48%. That gain was a result of judge error. Three judges compared to 200 responses marked by humans (Table 1) showed a 69-point swing in reported attack success on the same outputs; the local judge scored 128/162 true refusals as attacks (worse than always answering "refused", 81%). With a valid judge on 120 held-out behaviors, the trained attacker was 12.5% compared to 11.7% ($p = 1.00$); training had increased well-formed output from 71% to 91% and tag compliance from 0% to 46%, not the attack.

Table 1: Three judges on the same 200 responses. "Reported ASR" is what a paper with that judge would print; human-labelled truth is 19%.

Judge	Acc.	κ	Prec.	Rec.	False pos.	Reported ASR
`llama3.2`	36%	0.09	23%	100%	128	**83%**
Refusal heuristic	94%	0.83	76%	100%	12	25%
`gpt-4o-mini`	93%	0.75	93%	68%	2	**14%**

References

[1] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *ICML*, 2024. arXiv:2402.04249.

[2] P. Jha, A. Arora, and V. Ganesh. LLMStinger: Jailbreaking LLMs using RL fine-tuned LLMs. arXiv:2411.08862, 2024.

[3] X. Yang, X. Tang, J. Han, and S. Hu. The dark side of trust: Authority citation-driven jailbreak attacks on large language models. arXiv:2411.11407, 2024.

[4] L. Schwinn, M. Ladenburger, T. Beyer, M. Mofakhami, G. Gidel, and S. Günnemann. A coin flip for safety: LLM judges fail to reliably measure adversarial robustness. arXiv:2603.06594, 2026.

[5] Y. Gao. How reliable is your jailbreak judge? Calibration and adversarial robustness of automated ASR scoring. arXiv:2606.25487, 2026.

[6] A. Cartagena and A. Teixeira. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents. arXiv:2602.16943, 2026.

[7] I. Wicaksono, Z. Wu, R. Patel, T. King, A. Koshiyama, and P. Treleaven. Mind the gap: Evaluating model- and agentic-level vulnerabilities in LLMs with action graphs. arXiv:2509.04802, 2025.

[8] X. Chen, J. Zhang, and F. Tramèr. Learning to inject: Automated prompt injection via reinforcement learning. arXiv:2602.05746, 2026.

[9] X. Lin, Y. Yang, D. Guo, S. A. Nale, C. Fleming, and G. Cheng. Context-fractured decomposition attacks on tool-using LLM agents: Exploiting artifact provenance gaps. arXiv:2606.09084, 2026.

[10] T. Cao, Y. Chen, H. Cao, Y. Li, K. Le, T. Nguyen, Y. Li, Y. He, Y. Liu, S. Yan, and B. Hooi. WARD: Adversarially robust defense of web agents against prompt injections. *ICML 2026 Workshop on Agents in the Wild*. arXiv:2605.15030, 2026.